

INDIGO / VAJRA

AI Powered Threat Intelligence & Risk Analysis Dashboard

Comprehensive Technical Documentation

Document Version:	2.0 - Comprehensive Edition
Generated:	August 03, 2026 at 19:11:05
Document Type:	Complete Technical Specification
Page Count:	30+ Pages
Scope:	Full Stack Application Documentation
Technologies:	Next.js, FastAPI, PostgreSQL, Redis, WebSocket
Security Level:	Enterprise-Grade

Table of Contents

1. Executive Summary	Page 3
2. Project Overview	Page 4
3. System Architecture	Page 5
4. Backend Documentation	Page 6
5. Frontend Documentation	Page 7
6. Complete API Documentation	Page 8
7. Database Schema	Page 9
8. Services & Integrations	Page 10
9. Security Implementation	Page 11
10. Component Architecture	Page 12
11. Deployment Guide	Page 13
12. Development Workflow	Page 14
13. Testing & Quality Assurance	Page 15
14. Performance Optimization	Page 16
15. Monitoring & Logging	Page 17
16. Troubleshooting Guide	Page 18
17. Future Enhancements	Page 19

1. Executive Summary

INDIGO/VAJRA represents a state-of-the-art cybersecurity intelligence platform designed to provide real-time threat monitoring, ransomware tracking, and comprehensive risk analysis capabilities. The platform leverages modern web technologies and artificial intelligence to deliver actionable security intelligence to security professionals and organizations. **Core Value Proposition:** The platform addresses critical cybersecurity challenges by providing unified threat intelligence from multiple sources, real-time monitoring capabilities, and automated risk assessment. It serves as a central hub for security teams to monitor, analyze, and respond to emerging threats efficiently. **Key Differentiators:** • Real-time threat intelligence aggregation from multiple sources • Advanced domain security analysis with vulnerability assessment • Interactive global attack visualization • Comprehensive ransomware incident tracking • Executive-level risk summaries and reporting • WebSocket-based real-time updates • Enterprise-grade security architecture • Scalable microservices-inspired architecture **Target Users:** • Security Operations Centers (SOC) • Threat Intelligence Teams • Cybersecurity Analysts • Risk Management Professionals • Executive Leadership • Incident Response Teams

Platform Capabilities

Capability	Description	Status
Threat Intelligence	Multi-source threat data aggregation	Production Ready
Ransomware Tracking	Real-time ransomware incident monitoring	Production Ready
Domain Analysis	Comprehensive domain security assessment	Production Ready
Attack Visualization	Interactive global attack mapping	Production Ready
Report Generation	Automated PDF report creation	Production Ready
Real-time Updates	WebSocket-based live data streaming	Production Ready
User Management	Role-based access control	Production Ready
API Integration	RESTful API with comprehensive endpoints	Production Ready

2. Project Overview

Project Mission: To provide organizations with a comprehensive, real-time cybersecurity intelligence platform that enables proactive threat detection, risk assessment, and incident response through advanced data aggregation and analysis. **Technical Vision:** The platform is built on a modern technology stack emphasizing performance, scalability, and maintainability. The architecture follows best practices for separation of concerns, modular design, and enterprise security standards.

Complete Technology Stack

Layer	Technology	Version	Purpose
Frontend Framework	Next.js	15.5.20	React framework with App Router
React Library	React	19.0.0	UI component library
Language	TypeScript	5.x	Type-safe JavaScript
Styling	TailwindCSS	3.x	Utility-first CSS framework
State Management	Zustand	4.x	Client-side state management
Charts	Recharts	2.x	Data visualization library
Maps	React Simple Maps	3.x	Interactive geographic maps
Animations	Framer Motion	10.x	Animation library
Icons	Lucide React	0.x	Icon library
Forms	React Hook Form	7.x	Form management
Validation	Zod	3.x	Schema validation
HTTP Client	Axios	1.x	HTTP requests
Backend Framework	FastAPI	0.x	Python web framework
Database	PostgreSQL	15+	Relational database
Caching	Redis	7+	In-memory data store
ORM	SQLAlchemy	2.x	Database ORM
Authentication	JWT	HS256	Token-based auth
Task Scheduling	APScheduler	3.x	Background tasks
Real-time	WebSockets	Native	Live updates
PDF Generation	ReportLab	4.x	PDF creation
Containerization	Docker	24.x	Container management
Process Management	Docker Compose	2.x	Multi-container orchestration

Core Features

Threat Intelligence Module: • Real-time threat data aggregation from multiple sources • Threat score calculation and trending • IOC (Indicators of Compromise) management • Threat actor profiling and tracking • Industry-specific threat analysis

Ransomware Monitoring: • Live ransomware incident tracking • Ransomware group intelligence • Victim organization analysis • Payment and decryption tracking • Historical incident database

Domain Security Analysis: • Comprehensive domain risk assessment • DNS record analysis and monitoring • SSL/TLS certificate validation • WHOIS data extraction and analysis • Vulnerability assessment via NVD • Malware detection integration • Security posture scoring

Global Attack Visualization: • Interactive world map with attack origins • Real-time attack animation • Country-specific threat statistics • Attack type categorization • Historical attack patterns

Executive Reporting: • Automated PDF report generation • Executive summary creation • Risk counter dashboards • Trend analysis and forecasting • Customizable report templates

3. System Architecture

Architecture Overview: The INDIGO/VAJRA platform follows a modern microservices-inspired architecture with clear separation of concerns across multiple layers. The system is designed for scalability, maintainability, and security. **Architecture Layers:**

- 1. Presentation Layer (Frontend):**
 - Next.js 15 with App Router for server-side rendering
 - React 19 components for modular UI development
 - TypeScript for type safety and better developer experience
 - TailwindCSS for utility-first styling approach
 - Zustand for efficient client-side state management
 - Recharts for comprehensive data visualization
 - React Simple Maps for geographic data representation
- 2. Application Layer (Backend):**
 - FastAPI framework for high-performance API endpoints
 - Modular router architecture for organized API structure
 - Pydantic schemas for request/response validation
 - JWT-based authentication and authorization
 - Rate limiting and security middleware
 - Comprehensive error handling and logging
- 3. Business Logic Layer (Services):**
 - Third-party API integration services
 - Data processing and transformation logic
 - Threat intelligence aggregation
 - Domain analysis algorithms
 - Report generation services
 - Activity logging and auditing
- 4. Data Access Layer:**
 - SQLAlchemy ORM for database operations
 - Redis caching for performance optimization
 - Database connection pooling
 - Transaction management
 - Data validation and query optimization
- 5. Data Storage Layer:**
 - PostgreSQL 15+ for relational data storage
 - Redis 7+ for high-performance caching
 - File system for generated reports
 - Structured data models and relationships
- 6. Integration Layer:**
 - Third-party API clients (AlienVault, VirusTotal, etc.)
 - WebSocket manager for real-time communication
 - Background task scheduler
 - Event-driven architecture components

Data Flow Architecture

Request Flow:

1. User interacts with Next.js frontend
2. Frontend makes API calls to FastAPI backend
3. Backend validates requests via Pydantic schemas
4. Business logic processes requests in service layer
5. Data access layer queries PostgreSQL/Redis
6. Third-party APIs are called for additional data
7. Results are aggregated and transformed
8. Response is returned to frontend via JSON
9. Frontend updates UI with new data
10. WebSocket broadcasts real-time updates to connected clients

Background Task Flow:

1. APScheduler triggers scheduled tasks
2. Tasks fetch data from external sources
3. Data is processed and stored in database
4. WebSocket manager broadcasts updates
5. Connected clients receive real-time notifications
6. Cache is updated for performance

4. Backend Documentation

Backend Architecture: The backend is built on FastAPI, providing a high-performance, modern Python web framework with automatic API documentation, type hints, and async support. The architecture follows modular design principles with clear separation of concerns.

Core Components:

- main.py - Application Entry Point:**
 - FastAPI application initialization
 - Router registration and configuration
 - Middleware setup (CORS, security, rate limiting)
 - WebSocket integration
 - Background scheduler startup
 - Database initialization
 - Lifespan management (startup/shutdown)
- routes/ - API Endpoint Definitions:**
 - auth.py - Authentication and authorization endpoints
 - dashboard.py - Dashboard data aggregation
 - threat.py - Threat intelligence endpoints
 - ransomware.py - Ransomware incident management
 - news.py - Cyber threat news feed
 - reports.py - PDF report generation
 - domain.py - Domain risk analysis
 - companies.py - Company and domain management
 - alerts.py - Critical alerts management
 - threat_actors.py - Threat actor profiling
 - industries.py - Industry-specific analysis
 - notifications.py - User notifications
- services/ - Business Logic Layer:**
 - alienvault_service.py - AlienVault OTX integration
 - abuseipdb_service.py - IP reputation analysis
 - virustotal_service.py - Malware scanning
 - nvd_service.py - Vulnerability database
 - gridinsoft_service.py - Malware analysis
 - ssl_labs_service.py - SSL certificate analysis
 - pdf_service.py - PDF report generation
 - ransomware_service.py - Ransomware data management
 - dashboard_service.py - Dashboard aggregation
 - activity_logger.py - User activity tracking
- models/ - Database Models:**
 - user.py - User account model
 - alert.py - Security alert model
 - threat_feed.py - Threat intelligence model
 - attack_map.py - Geographic attack data
 - ransomware_incident.py - Ransomware incident model
 - company.py - Company/domain model
 - And additional models for all data entities
- middleware/ - Security Middleware:**
 - rate_limit.py - API rate limiting
 - security.py - Security headers and protections
- websocket/ - Real-time Communication:**
 - websocket_manager.py - Connection management
 - websocket_routes.py - WebSocket endpoints
- scheduler/ - Background Tasks:**
 - scheduler.py - Task scheduling configuration

Complete API Routes Reference

Route Prefix	Purpose	Key Endpoints
/api/auth	Authentication	/register, /login, /refresh, /logout, /profile
/api/dashboard	Dashboard Data	/summary, /alerts, /attack-map
/api/threat-intelligence	Threat Intel	/, /trend, /actors, /industries
/api/ransomware	Ransomware	/, /stats, /groups
/api/news	Threat News	/, /latest
/api/reports	Report Generation	/threat, /ransomware, /executive
/api/domain-risk	Domain Analysis	/scan, /analyze

/api/companies	Company Management	/, /{id}, /{id}/analyze
/api/alerts	Alert Management	/, /{id}, /acknowledge
/ws/dashboard	WebSocket	Real-time dashboard updates

5. Frontend Documentation

Frontend Architecture: The frontend is built on Next.js 15 with the App Router, providing a modern React framework with server-side rendering, optimized performance, and excellent developer experience. The architecture emphasizes component reusability, type safety, and responsive design. **Key Frontend Technologies: Next.js 15 Features:** • App Router for file-based routing • Server Components for improved performance • Client Components for interactivity • API Routes for backend communication • Built-in optimization and caching • TypeScript support out of the box **React 19 Enhancements:** • Concurrent rendering for better UX • Automatic batching of state updates • Improved hooks and APIs • Better error handling **State Management (Zustand):** • Lightweight state management • Simple API with hooks • No provider wrapping needed • Excellent TypeScript support • DevTools integration **Styling (TailwindCSS):** • Utility-first CSS approach • Responsive design utilities • Dark mode support • Custom theme configuration • Optimized production builds **Frontend Structure: app/ - Next.js App Router:** • page.tsx - Main dashboard page • layout.tsx - Root layout with providers • globals.css - Global styles and Tailwind imports • login/ - Authentication page • companies/ - Company and domain analysis • domain-analysis/ - Deep domain security analysis • threat-intelligence/ - Threat intelligence dashboard • ransomware/ - Ransomware incident tracking • global-attacks/ - Global attack visualization • executive-summary/ - Executive risk summary • alerts/ - Critical alerts management • settings/ - Application settings **components/ - React Components:** • TopCards.tsx - Dashboard summary cards • ThreatMap.tsx - Interactive global attack map • AlertCards.tsx - Horizontal scrolling alerts • DomainDetails.tsx - Comprehensive domain analysis • Various chart components for data visualization • UI components (buttons, cards, modals, etc.) **services/ - API Services:** • dashboard.service.ts - Dashboard API calls • domain.service.ts - Domain analysis API • auth.service.ts - Authentication API • threat.service.ts - Threat intelligence API • ransomware.service.ts - Ransomware API **store/ - Zustand Stores:** • authStore.ts - Authentication state • dashboardStore.ts - Dashboard state • uiStore.ts - UI state management

Frontend Pages Reference

Route	Page Name	Purpose	Key Features
/	Dashboard	Main overview	Threat summary, attack map, alerts
/login	Login	Authentication	JWT login, form validation
/companies	Companies	Company analysis	Domain monitoring, risk assessment
/domain-analysis	Domain Analysis	Deep security analysis	DNS, SSL, WHOIS, vulnerabilities
/threat-intelligence	Threat Intel	Threat dashboard	IOC management, threat scores
/ransomware	Ransomware	Incident tracking	Live incidents, group analysis

/global-attacks	Global Attacks	Attack visualization	Interactive world map
/executive-summary	Executive Summary	Risk overview	High-level metrics, reports
/alerts	Alerts	Alert management	Critical alerts, acknowledgments
/settings	Settings	Configuration	User preferences, API keys

6. Complete API Documentation

API Architecture: The RESTful API is built on FastAPI, providing automatic OpenAPI documentation, type validation, and high performance. All endpoints follow REST principles and return JSON responses. **Authentication Flow:** 1. User registers via POST /api/auth/register 2. User logs in via POST /api/auth/login 3. Server returns JWT access token (30 min expiry) 4. Server returns JWT refresh token (7 days expiry) 5. Client includes access token in Authorization header 6. When access token expires, client uses refresh token 7. POST /api/auth/refresh generates new access token 8. POST /api/auth/logout invalidates tokens

Authentication Endpoints: **POST /api/auth/register** • Description: Register new user account • Request Body: { "username": "string", "email": "string", "password": "string" } • Response: { "access_token": "string", "refresh_token": "string", "user": {...} } • Status Codes: 201 (Created), 400 (Bad Request), 409 (Conflict)

POST /api/auth/login • Description: Authenticate user and receive tokens • Request Body: { "username": "string", "password": "string" } • Response: { "access_token": "string", "refresh_token": "string", "user": {...} } • Status Codes: 200 (OK), 401 (Unauthorized)

POST /api/auth/refresh • Description: Refresh access token using refresh token • Request Body: { "refresh_token": "string" } • Response: { "access_token": "string" } • Status Codes: 200 (OK), 401 (Unauthorized)

POST /api/auth/logout • Description: Logout user and invalidate tokens • Request Headers: Authorization: Bearer {access_token} • Response: { "message": "Successfully logged out" } • Status Codes: 200 (OK), 401 (Unauthorized)

GET /api/auth/profile • Description: Get current user profile • Request Headers: Authorization: Bearer {access_token} • Response: { "id": "int", "username": "string", "email": "string", ... } • Status Codes: 200 (OK), 401 (Unauthorized)

Dashboard

Endpoints: **GET /api/dashboard/summary** • Description: Get dashboard summary statistics • Response: { "total_attacks": "int", "active_threat_actors": "int", ... } • Status Codes: 200 (OK)

GET /api/dashboard/alerts • Description: Get critical security alerts • Query Params: limit (int), severity (string) • Response: [{ "id": "int", "title": "string", "severity": "string", ... }] • Status Codes: 200 (OK)

GET /api/dashboard/attack-map • Description: Get global attack map data • Response: [{ "country": "string", "attack_count": "int", "coordinates": [...] }] • Status Codes: 200 (OK)

Threat Intelligence Endpoints: **GET /api/threat-intelligence** • Description: Get threat intelligence data • Query Params: limit (int), offset (int) • Response: [{ "id": "int", "threat_type": "string", "severity": "string", ... }] • Status Codes: 200 (OK)

GET /api/threat-intelligence/trend • Description: Get threat trend data over time • Query Params: days (int) • Response: [{ "date": "string", "threat_count": "int", ... }] • Status Codes: 200 (OK)

GET /api/threat-intelligence/actors • Description: Get threat actor profiles • Response: [{ "id": "int", "name": "string", "activity_level": "string", ... }] • Status Codes: 200 (OK)

Ransomware Endpoints: **GET /api/ransomware** • Description: Get ransomware incidents • Query Params: limit (int), active_only (boolean) • Response: [{ "id": "int", "victim_name": "string", "group_name": "string", ... }] • Status Codes: 200 (OK)

GET /api/ransomware/stats • Description: Get ransomware statistics • Response: { "total_incidents": "int", "active_groups": "int", ... } • Status Codes: 200 (OK)

Domain Analysis Endpoints: **POST /api/domain-risk/scan** •

Description: Scan domain for security risks • Request Body: { "domain": "string" } •
Response: { "domain": "string", "risk_score": "int", "threats": [...], ... } • Status Codes: 200
(OK), 400 (Bad Request) **Report Endpoints: GET /api/reports/threat** • Description:
Generate threat intelligence PDF report • Query Params: format (pdf), date_range (string)
• Response: PDF file download • Status Codes: 200 (OK), 404 (Not Found) **GET**
/api/reports/ransomware • Description: Generate ransomware PDF report • Response:
PDF file download • Status Codes: 200 (OK), 404 (Not Found) **GET**
/api/reports/executive • Description: Generate executive summary PDF report •
Response: PDF file download • Status Codes: 200 (OK), 404 (Not Found)

7. Database Schema

Database Architecture: The platform uses PostgreSQL 15+ as the primary relational database, providing robust data storage, ACID compliance, and advanced querying capabilities. Redis 7+ is used as a caching layer for performance optimization and session management. **Database Technologies:** • PostgreSQL 15+ - Primary relational database • Redis 7+ - Caching and session storage • SQLAlchemy 2.x - Python ORM for database operations • Alembic - Database migration management **Core Database Tables:**

users table: • id (Primary Key) - Unique user identifier • username (Unique) - User username for login • email (Unique) - User email address • hashed_password - Bcrypt hashed password • is_active - Account status flag • is_admin - Administrator privilege flag • created_at - Account creation timestamp • updated_at - Last update timestamp • last_login - Last successful login timestamp

alerts table: • id (Primary Key) - Unique alert identifier • title - Alert title and summary • description - Detailed alert description • severity - Alert severity level (critical, high, medium, low) • source - Alert source (threat intel, ransomware, etc.) • status - Alert status (active, acknowledged, resolved) • created_at - Alert creation timestamp • acknowledged_at - Acknowledgment timestamp • user_id (Foreign Key) - User who acknowledged

threat_feeds table: • id (Primary Key) - Unique threat feed identifier • source - Threat intelligence source • threat_type - Type of threat (malware, phishing, etc.) • indicators - JSON field for IOCs • severity - Threat severity level • confidence - Confidence score (0-100) • created_at - Feed creation timestamp • updated_at - Last update timestamp

attack_map table: • id (Primary Key) - Unique attack identifier • country - Target country code • attack_count - Number of attacks • attack_types - JSON field for attack type breakdown • latitude - Geographic latitude • longitude - Geographic longitude • timestamp - Attack timestamp

ransomware_incidents table: • id (Primary Key) - Unique incident identifier • victim_name - Name of victim organization • group_name - Ransomware group responsible • attack_date - Date of attack • sector - Industry sector • country - Country of victim • description - Incident description • status - Incident status (active, resolved, ongoing) • ransom_demand - Ransom amount demanded • payment_status - Payment status • created_at - Record creation timestamp • updated_at - Last update timestamp

companies table: • id (Primary Key) - Unique company identifier • name - Company name • domain - Primary domain • industry - Industry sector • size - Company size • risk_score - Calculated risk score • last_analysis - Last analysis timestamp • monitoring_status - Monitoring status • created_at - Record creation timestamp • updated_at - Last update timestamp

domain_analysis table: • id (Primary Key) - Unique analysis identifier • company_id (Foreign Key) - Associated company • domain - Analyzed domain • dns_records - JSON field for DNS data • ssl_info - JSON field for SSL certificate data • whois_data - JSON field for WHOIS information • security_score - Calculated security score • vulnerabilities - JSON field for vulnerability data • threats - JSON field for detected threats • analysis_date - Analysis timestamp • created_at - Record creation timestamp

news table: • id (Primary Key) - Unique news identifier • title - News article title • content - Article content • source - News source • url - Article URL • published_at - Publication timestamp • created_at - Record creation

timestamp **reports table:** • id (Primary Key) - Unique report identifier • report_type - Type of report (threat, ransomware, executive) • title - Report title • generated_by - User who generated report • file_path - Path to generated PDF file • parameters - JSON field for report parameters • created_at - Report generation timestamp **activity_logs table:** • id (Primary Key) - Unique log identifier • user_id (Foreign Key) - User who performed action • action - Action performed • resource_type - Type of resource affected • resource_id - ID of affected resource • details - JSON field for additional details • ip_address - IP address of user • user_agent - User agent string • timestamp - Action timestamp

Database Relationships

Key Relationships: • users → alerts (one-to-many) - Users can acknowledge multiple alerts • users → reports (one-to-many) - Users can generate multiple reports • users → activity_logs (one-to-many) - Users have multiple activity logs • companies → domain_analysis (one-to-many) - Companies have multiple domain analyses • ransomware_groups → ransomware_incidents (one-to-many) - Groups claim multiple incidents • threat_feeds → alerts (one-to-many) - Threat feeds generate multiple alerts

8. Services & Integrations

Services Architecture: The services layer contains all business logic and third-party integrations. Each service is designed to be modular, testable, and maintainable. Services handle data transformation, API communication, and complex business operations.

Third-Party Integrations:

- AlienVault OTX Service:** • Purpose: Fetch threat intelligence indicators • API: AlienVault Open Threat Exchange • Data: IOCs, threat pulses, malware samples • Rate Limit: 20 requests per minute • Authentication: API Key • Usage: Threat intelligence aggregation, IOC enrichment
- AbuseIPDB Service:** • Purpose: IP reputation and abuse reporting • API: AbuseIPDB REST API • Data: IP confidence scores, abuse reports, geolocation • Rate Limit: 1000 requests per day • Authentication: API Key • Usage: IP risk assessment, threat correlation
- VirusTotal Service:** • Purpose: Malware and URL analysis • API: VirusTotal REST API • Data: File scans, URL analysis, detection ratios • Rate Limit: 500 requests per day • Authentication: API Key • Usage: Malware detection, file reputation
- NVD Service:** • Purpose: National Vulnerability Database integration • API: NIST NVD API v2.0 • Data: CVE records, CVSS scores, vulnerability details • Rate Limit: 50 requests per rolling 30 seconds • Authentication: API Key (NVD API Key) • Usage: Vulnerability assessment, security scoring
- Gridinsoft Service:** • Purpose: Malware analysis and detection • API: Gridinsoft Anti-Malware API • Data: Malware signatures, threat classification • Rate Limit: 100 requests per hour • Authentication: API Key • Usage: Domain malware analysis, threat detection
- SSL Labs Service:** • Purpose: SSL/TLS certificate analysis • API: SSL Labs Assessment API • Data: Certificate details, SSL grade, vulnerabilities • Rate Limit: 10 assessments per day • Authentication: None (public API) • Usage: SSL certificate validation, security scoring
- URLScan.io Service:** • Purpose: URL scanning and analysis • API: URLScan.io REST API • Data: Scan results, screenshot, technologies detected • Rate Limit: 1000 requests per day • Authentication: API Key • Usage: URL analysis, threat detection
- Internal Services:**
- PDF Service:** • Purpose: Generate PDF reports • Technology: ReportLab Python library • Features: Custom styling, tables, charts, headers/footers • Output: Professional PDF documents • Usage: Executive reports, threat intelligence reports
- Dashboard Service:** • Purpose: Aggregate dashboard data • Operations: Data aggregation, statistics calculation • Data Sources: Multiple database tables, external APIs • Caching: Redis caching for performance • Usage: Main dashboard data provider
- Domain Analysis Service:** • Purpose: Comprehensive domain security analysis • Operations: DNS analysis, SSL validation, WHOIS lookup • Integrations: Multiple external APIs • Scoring: Custom security score calculation • Usage: Domain risk assessment, security monitoring
- Ransomware Service:** • Purpose: Ransomware incident management • Operations: Data fetching, incident tracking, statistics • Data Source: threat-ransomware.live • Scheduling: Background task every 15 minutes • Usage: Ransomware monitoring, incident analysis
- Activity Logger Service:** • Purpose: Track user activities • Operations: Log creation, audit trail maintenance • Data: User actions, timestamps, IP addresses • Usage: Security auditing, compliance
- WebSocket Manager:** • Purpose: Real-time connection management • Operations: Connection handling, message

broadcasting • Features: Connection pooling, error handling • Usage: Real-time dashboard updates, live notifications

Background Tasks Configuration

Task Name	Frequency	Purpose	Data Source
Ransomware Data Fetch	Every 15 minutes	Fetch ransomware incidents	threat-ransomware.live
Threat Intel Refresh	Every 30 minutes	Update threat indicators	AlienVault OTX
Dashboard Stats Refresh	Every 5 minutes	Refresh dashboard metrics	Database aggregation
Activity Logging	Continuous	Track user activities	Application events
Cache Refresh	Every 10 minutes	Update Redis cache	Database queries
SSL Certificate Check	Every hour	Validate SSL certificates	SSL Labs API

9. Security Implementation

Security Architecture: The platform implements enterprise-grade security measures across all layers, following industry best practices and security standards. Security is integrated into every component, from authentication to data encryption.

Authentication & Authorization: JWT Token System:

- Access Tokens: 30-minute expiration for security
- Refresh Tokens: 7-day expiration for convenience
- Token Storage: HttpOnly cookies for XSS protection
- Token Refresh: Automatic token renewal mechanism
- Token Revocation: Server-side token invalidation
- Algorithm: HS256 for token signing

Password Security:

- Hashing Algorithm: Bcrypt with salt rounds
- Salt Rounds: 12 for optimal security/performance balance
- Password Requirements: Minimum 8 characters, mixed case
- Password Reset: Secure reset flow with email verification
- Password History: Prevents reuse of recent passwords

API Security: Rate Limiting:

- Implementation: slowapi library
- Default Limit: 100 requests per minute per IP
- Authenticated Limit: 200 requests per minute per user
- Sensitive Endpoints: Stricter limits (10 requests per minute)
- Response Headers: Rate limit information in headers
- Bypass Mechanism: Admin role bypass for operations

CORS Configuration:

- Development: Allow all origins for flexibility
- Production: Configured whitelist of allowed origins
- Methods: GET, POST, PUT, DELETE, OPTIONS
- Headers: Authorization, Content-Type, Accept
- Credentials: Enabled for cookie-based auth

Input Validation:

- Schema Validation: Pydantic schemas for all inputs
- Type Checking: Strict type enforcement
- Length Limits: Maximum length constraints
- Format Validation: Email, URL, and custom formats
- SQL Injection Prevention: ORM parameterized queries
- XSS Protection: Input sanitization and output encoding

Security Headers:

- Content-Security-Policy: Strict CSP configuration
- X-Frame-Options: DENY to prevent clickjacking
- X-Content-Type-Options: nosniff for MIME type protection
- Strict-Transport-Security: HSTS for HTTPS enforcement
- X-XSS-Protection: XSS attack prevention
- Referrer-Policy: Strict referrer policy

Data Security: Encryption:

- Passwords: Bcrypt hashing with salt
- Sensitive Data: AES-256 encryption at rest
- Transit: TLS 1.3 for all communications
- Database: PostgreSQL encryption at rest
- Redis: TLS for Redis connections

Session Management:

- Storage: Redis for session data
- Expiration: Automatic session cleanup
- Secure Cookies: HttpOnly and Secure flags
- Session Fixation: Regeneration on login
- Concurrent Sessions: Limit per user

Monitoring & Auditing: Activity Logging:

- User Actions: All user activities logged
- API Calls: Request/response logging
- Failed Attempts: Login failure tracking
- IP Tracking: Source IP logging
- User Agent: Client information logging
- Timestamp: Precise event timing

Security Events:

- Brute Force Detection: Multiple failed logins
- Anomaly Detection: Unusual activity patterns
- Data Breach Alerts: Sensitive data access
- Privilege Escalation: Admin action monitoring
- API Abuse: Rate limit violation tracking

10. Component Architecture

Component Design Principles: The frontend follows component-based architecture with React, emphasizing reusability, maintainability, and performance. Components are organized by functionality and follow single responsibility principles. **Component**

Hierarchy: **Layout Components:** • Root Layout (layout.tsx) - Application wrapper with providers • Dashboard Layout - Main dashboard structure • Auth Layout - Authentication page layout • Admin Layout - Admin panel layout **Page Components:** • Dashboard Page (page.tsx) - Main dashboard • Login Page - Authentication interface • Companies Page - Company management • Domain Analysis Page - Domain security analysis • Threat Intelligence Page - Threat dashboard • Ransomware Page - Incident tracking • Global Attacks Page - Attack visualization • Executive Summary Page - Risk overview **UI**

Components: • TopCards - Dashboard summary cards • ThreatMap - Interactive attack map • AlertCards - Scrolling alert cards • DomainDetails - Domain analysis display • Chart Components - Various data visualizations • Table Components - Data tables with sorting/filtering • Modal Components - Dialog overlays • Form Components - Input forms with validation • Button Components - Action buttons with states • Card Components - Content containers

State Management: Zustand Stores: • authStore - User authentication state • dashboardStore - Dashboard data state • uiStore - UI state (modals, sidebars, themes) • alertStore - Alert management state • companyStore - Company data state **Component Communication:** • Props Down - Parent to child data flow • Events Up - Child to parent event flow • Context API - Global state sharing • Zustand - Cross-component state management • Custom Hooks - Reusable state logic

Performance Optimization: React Optimization: • React.memo - Component memoization • useMemo - Expensive calculation caching • useCallback - Function reference stability • Code Splitting - Lazy loading components • Server Components - Server-side rendering • Image Optimization - Next.js Image component

11. Deployment Guide

Deployment Architecture: The platform supports multiple deployment strategies, from local development to production cloud deployments. Docker containerization ensures consistency across environments.

Docker Deployment: Prerequisites: • Docker 24.x or higher • Docker Compose 2.x or higher • 4GB RAM minimum • 20GB disk space minimum

Quick Start: 1. Clone the repository 2. Copy environment files: - cp backend/.env.example backend/.env - cp frontend/.env.example frontend/.env.local 3. Configure environment variables 4. Build and start services: - docker-compose up -d --build 5. Access the application: - Frontend: http://localhost:3000 - Backend: http://localhost:8000 - API Docs: http://localhost:8000/docs

Docker Compose Services: • frontend - Next.js frontend application • backend - FastAPI backend application • postgres - PostgreSQL database • redis - Redis caching layer • nginx - Reverse proxy (production)

Production Deployment: Environment Configuration: • DATABASE_URL: Production PostgreSQL connection • REDIS_URL: Production Redis connection • SECRET_KEY: Strong random secret key • NEXT_PUBLIC_API_URL: Production API URL • NEXT_PUBLIC_WS_URL: Production WebSocket URL • API Keys: Production API keys for all services

Security

Considerations: • Use strong, unique secrets • Enable HTTPS with SSL certificates • Configure firewall rules • Enable database backups • Set up monitoring and alerting • Use environment-specific configurations

Cloud Deployment Options:

AWS Deployment: • EC2 for application servers • RDS for PostgreSQL database • ElastiCache for Redis • S3 for static file storage • CloudFront for CDN • Load Balancer for traffic distribution

DigitalOcean Deployment: • Droplets for application servers • Managed PostgreSQL database • Managed Redis • Spaces for object storage • Load Balancer for high availability

Local Development: Backend Setup: 1. Create virtual environment: - python -m venv venv - source venv/bin/activate 2. Install dependencies: - pip install -r requirements.txt 3. Configure environment: - cp .env.example .env 4. Run database migrations (if using Alembic): - alembic upgrade head 5. Start the server: - uvicorn main:app --reload

Frontend Setup: 1. Install dependencies: - npm install 2. Configure environment: - cp .env.example .env.local 3. Start development server: - npm run dev

12. Development Workflow

Development Process: The project follows a structured development workflow to ensure code quality, maintainability, and collaboration.

Git Workflow:

- main - Production-ready code
- develop - Integration branch
- feature/* - Feature branches
- bugfix/* - Bug fix branches
- hotfix/* - Emergency fixes

Branching Strategy:

1. Create feature branch from develop
2. Implement changes with commits
3. Test locally before pushing
4. Create pull request to develop
5. Code review and approval
6. Merge to develop
7. Deploy to staging environment
8. Test staging deployment
9. Merge develop to main
10. Deploy to production

Code Standards:

Python Standards:

- PEP 8 compliance
- Type hints for functions
- Docstrings for modules and functions
- Maximum line length: 100 characters
- meaningful variable names
- Error handling with try/except

TypeScript Standards:

- Strict mode enabled
- Interface definitions for data structures
- Type annotations for all functions
- No any types unless necessary
- Proper null checks
- React functional components

Testing Strategy:

Backend Testing:

- Unit tests for services
- Integration tests for API endpoints
- Database tests with test database
- Mock external API calls
- Coverage target: 80%+

Frontend Testing:

- Component tests with React Testing Library
- Integration tests for user flows
- E2E tests with Playwright
- Visual regression tests
- Coverage target: 70%+

Code Review Process:

1. Self-review before PR
2. Automated CI/CD checks
3. Peer review by team member
4. Address review comments
5. Approval required for merge
6. Squash merge to main

CI/CD Pipeline:

- Automated testing on push
- Code quality checks
- Security scanning
- Build verification
- Deployment to staging
- Manual approval for production

13. Testing & Quality Assurance

Testing Strategy: Comprehensive testing ensures reliability, security, and performance. The project employs multiple testing methodologies at different levels of the application stack.

Backend Testing:

- Unit Testing:** • Framework: pytest • Scope: Individual functions and methods • Mocking: unittest.mock for external dependencies • Coverage: Service layer, utilities, helpers • Target: 80%+ code coverage
- Integration Testing:** • Framework: pytest with test database • Scope: API endpoints and database operations • Database: In-memory SQLite for speed • API Testing: TestClient from FastAPI • Coverage: All API endpoints

Frontend Testing:

- Component Testing:** • Framework: React Testing Library • Scope: Individual React components • Mocking: MSW for API mocking • Coverage: UI components, hooks, utilities • Target: 70%+ code coverage
- Integration Testing:** • Framework: Playwright • Scope: User flows and interactions • Browser: Chromium, Firefox, WebKit • Coverage: Critical user paths • Target: All major user flows

Testing Best Practices:

- Test isolation (no dependencies between tests)
- Descriptive test names
- Arrange-Act-Assert pattern
- Mock external dependencies
- Test edge cases and error conditions
- Regular test maintenance
- Automated test execution in CI/CD

Quality Assurance:

Code Quality Tools:

- ESLint - JavaScript/TypeScript linting
- Prettier - Code formatting
- Black - Python code formatting
- Flake8 - Python linting
- mypy - Python type checking

Security Testing:

- OWASP ZAP - Security scanning
- Dependency scanning
- Vulnerability checks
- Secret scanning
- Credential detection
- Penetration testing
- Manual security review

14. Performance Optimization

Performance Strategy: The platform implements multiple optimization techniques to ensure fast response times, efficient resource utilization, and excellent user experience.

Frontend Performance: Code Optimization: • Code splitting - Lazy load components • Tree shaking - Remove unused code • Minification - Reduce bundle size • Compression - Gzip/Brotli compression • Image optimization - WebP format, lazy loading • Font optimization - Subset fonts, preload critical fonts

Rendering Optimization: • Server Components - Reduce client JavaScript • Static Generation - Pre-render static content • Incremental Static Regeneration - Update static content • Memoization - React.memo, useMemo, useCallback • Virtual scrolling - Large lists optimization

Backend Performance: Database Optimization: • Indexing - Strategic database indexes • Query optimization - Efficient SQL queries • Connection pooling - Reuse database connections • Caching - Redis for frequent queries • Pagination - Limit result sets • N+1 query prevention - Eager loading

API Optimization: • Async operations - Non-blocking I/O • Response compression - Gzip compression • Rate limiting - Prevent abuse • Pagination - Large data sets • Field selection - Partial responses • Batch operations - Reduce round trips

Caching Strategy: Redis Caching: • Dashboard data - 5 minute TTL • Threat intelligence - 30 minute TTL • User sessions - 7 day TTL • API responses - Configurable TTL • Cache invalidation - Smart cache updates

CDN Usage: • Static assets - CDN distribution • Images - CDN caching • Fonts - CDN delivery • JavaScript bundles - CDN caching

Monitoring Performance: • Response time tracking • Database query monitoring • Cache hit/miss ratios • Error rate monitoring • User experience metrics

15. Monitoring & Logging

Monitoring Strategy: Comprehensive monitoring ensures system health, performance tracking, and issue detection. The platform implements monitoring at multiple levels.

Application Logging: Log Levels: • DEBUG - Detailed diagnostic information • INFO - General informational messages • WARNING - Warning messages for potential issues • ERROR - Error messages for failures • CRITICAL - Critical system failures **Log Content:** • Timestamp - Precise event timing • Level - Log severity • Module - Source module/component • Message - Descriptive log message • User ID - Associated user (if applicable) • Request ID - Request correlation • Error details - Stack traces for errors

Performance Monitoring: Key Metrics: • Response times - API endpoint performance • Database query times - Database performance • Cache hit ratios - Caching effectiveness • Error rates - System reliability • Resource utilization - CPU, memory, disk • Network traffic

- Bandwidth usage **Health Checks:** • /health - Basic health endpoint • Database connectivity - Database health • Redis connectivity - Cache health • External API status - Third-party service health • Disk space - Storage availability **Alerting:** • Error rate thresholds - Alert on high error rates • Response time alerts - Alert on slow responses • Resource alerts - Alert on high resource usage • Security alerts - Alert on security events • Availability alerts - Alert on service downtime

Log Management: • Centralized logging - Aggregate logs from all services • Log rotation - Manage log file sizes • Log retention - Keep logs for specified period • Log analysis - Search and analyze logs • Log export - Export logs for analysis

16. Troubleshooting Guide

Common Issues and Solutions: This section provides solutions to common issues that may be encountered during development, deployment, or operation of the platform.

Development Issues:

- Backend won't start:** • Check Python version (3.11+ required) • Verify virtual environment is activated • Ensure all dependencies are installed • Check database connection string in .env • Verify port 8000 is not in use • Check for syntax errors in code
- Frontend won't start:** • Check Node.js version (20+ required) • Verify dependencies are installed (npm install) • Check environment variables in .env.local • Verify port 3000 is not in use • Clear Next.js cache (rm -rf .next)

Database connection issues: • Verify PostgreSQL is running • Check connection string in .env • Ensure database exists • Verify user credentials • Check network connectivity • Review PostgreSQL logs

Redis connection issues: • Verify Redis is running • Check Redis connection string • Ensure Redis is accessible • Check Redis logs for errors • Verify Redis configuration

API Issues:

- External API failures:** • Check API key validity • Verify API rate limits • Check API service status • Review error messages • Check network connectivity • Verify request format
- Authentication failures:** • Verify JWT secret key • Check token expiration • Verify user credentials • Review authentication logs • Check token refresh logic

Performance Issues:

- Slow response times:** • Check database query performance • Review caching effectiveness • Check for N+1 queries • Review external API call times • Check resource utilization • Review code for inefficiencies
- Memory issues:** • Check for memory leaks • Review caching strategy • Check for large data sets • Review database connection pooling • Monitor memory usage patterns

Deployment Issues:

- Docker build failures:** • Check Dockerfile syntax • Verify base image availability • Review build logs for errors • Check network connectivity • Verify disk space availability
- Container startup failures:** • Review container logs • Check environment variables • Verify service dependencies • Check resource limits • Review health check configuration

17. Future Enhancements

Planned Enhancements: The platform continues to evolve with planned enhancements to improve functionality, performance, and user experience.

Short-term Enhancements (3-6 months):

- Feature Additions:** • Advanced threat correlation algorithms • Machine learning-based threat prediction • Mobile application (iOS/Android) • Advanced reporting templates • Custom dashboard configurations • API rate limit management UI • Bulk domain analysis • Scheduled report generation
- Performance Improvements:** • Database query optimization • Advanced caching strategies • CDN integration for static assets • Image optimization pipeline • Code splitting improvements
- Security Enhancements:** • Multi-factor authentication • Advanced role-based access control • API key management system • Advanced audit logging • Security compliance reporting

Medium-term Enhancements (6-12 months):

- Advanced Analytics:** • Threat trend analysis • Predictive analytics • Custom alerting rules • Advanced visualization options • Data export capabilities
- Integration Expansions:** • Additional threat intelligence sources • SIEM system integration • Ticket system integration • Custom webhook integrations • API marketplace for third-party tools
- User Experience:** • Advanced user onboarding • Interactive tutorials • Custom themes and branding • Advanced search capabilities • Collaboration features

Long-term Enhancements (12+ months):

- Platform Expansion:** • Multi-tenant architecture • White-label solution • API marketplace • Plugin system • Community features
- Advanced Capabilities:** • AI-powered threat analysis • Automated incident response • Digital twin integration • Blockchain-based verification • Quantum-resistant cryptography
- Infrastructure:** • Edge computing deployment • Advanced CDN integration • Global load balancing • Disaster recovery automation • Compliance automation

Conclusion

This comprehensive documentation provides a complete overview of the INDIGO/VAJRA platform, covering all aspects from architecture to deployment. The platform represents a state-of-the-art cybersecurity intelligence solution that continues to evolve with emerging threats and technologies.

Key Takeaways:

- Modern technology stack with Next.js and FastAPI
- Comprehensive security implementation
- Real-time threat intelligence capabilities
- Scalable microservices-inspired architecture
- Enterprise-grade performance and reliability
- Extensive third-party integrations
- Professional reporting capabilities
- Active development and enhancement roadmap

Support and Maintenance: The platform is actively maintained with regular updates, security patches, and feature enhancements. Comprehensive support is available through documentation, issue tracking, and direct

communication channels. **Next Steps:** 1. Review this documentation thoroughly 2. Set up local development environment 3. Explore the API documentation 4. Review the codebase structure 5. Set up monitoring and logging 6. Plan deployment strategy 7. Establish development workflow 8. Begin feature development or customization

End of Comprehensive Documentation

Document Version: 2.0 - Comprehensive Edition

Generated on August 03, 2026 at 19:11:05

© 2026 INDIGO/VAJRA Project Team - All Rights Reserved